NBA Game Analysis Using Prediction Market Data

By Bradley Hofmeister, Ryan McCurry, Chris Halvonik

GitHub Link: https://github.com/brad-hof/Milestone2/tree/main

Introduction

The primary goal of this project is to investigate whether the prediction market Kalshi provides useful information for predicting NBA game outcomes beyond what traditional basketball statistics can provide. We built a machine-learning pipeline that combines NBA game and team statistics with Kalshi prediction-market information and evaluates how well each source predicts the winner of an NBA game. We compared models based on traditional NBA statistics, models based only on Kalshi market information, and models that combine both. This helped us determine whether Kalshi provides incremental predictive value and whether incorporating market information can improve NBA game predictions.

This matters as prediction markets grow as a source of sports forecasting. If market prices already capture what the box score shows, analysts and teams building win-probability models can treat the market as a strong baseline, and if they don't, it points to moments in a game when markets are slow to react. We chose this project out of a shared interest in sports analytics and the recent growth of regulated prediction markets like Kalshi, whose public API provides minute-level price history for every game.

A secondary goal was examining how the predictive performance of Kalshi changes from tipoff through the game. Since prediction-market prices change as new information becomes available, we evaluated Kalshi probabilities at multiple points during each game to determine whether they become increasingly accurate as the game progresses. We also looked at whether Kalshi's accuracy differs between closely matched games and games with a clear favorite.

The supervised methods used include logistic regression, random forest, and k-nearest neighbors. The unsupervised methods applied K-means and agglomerative hierarchical clustering to game-state features across pregame and in-game checkpoints. Unlike prior work, which studied either pre-game market prices or in-game statistics on their own, our project tracks Kalshi prices through each game and compares them directly against box score data at every checkpoint.

In the supervised analysis, no model meaningfully outperformed the Kalshi price, and adding NBA stats never improved test-set predictions, which suggests the market already reflects the score and each team's recent form. Results also show that Kalshi's forecasts become more accurate as games progress, while K-means consistently favors two clusters with limited separation. In addition, comparing features with and without market probabilities also reveals broadly similar clustering structures.

Related Work

We identified three references that, in combination, informed our project's objectives. The first is a research paper titled "When Do Markets Fully Process Public Information? Evidence from Real-Time Prediction Markets" (Angelini & De Angelis, 2026), which uses Kalshi's implied win probability for NBA games to study how the forecast accuracy changes in the 24 hours before the game begins. This closely relates to our second research question but is limited to the pre-game window and does not take into account actual NBA statistics during the game itself.

The second reference is another research paper titled “Bayesian estimation of in-game home team win probability for National Basketball Association games” (Maddox et al., 2022), which models in-game win probability using elapsed time and statistics throughout an NBA game. However, this reference does not utilize prediction markets in its model, and only references traditional statistics.

A third reference is the paper titled “Prediction Markets” (Wolfers & Zitzewitz, 2004), which studied economic theory behind prediction markets by finding prediction markets produce relatively accurate forecasts and can outperform traditional forecasting models. Our project builds on this by testing that claim directly for NBA games, measuring whether game statistics add anything beyond the market price at multiple points during each game.

This project builds on these three areas by tracking Kalshi’s implied win probabilities throughout the NBA games and comparing them with the traditional NBA statistics. In addition, we used clustering to group games by their characteristics and examine how performance differs across groups. This combination of real time market data with NBA statistics shows how market expectations align with game outcomes and how this relationship changes across different game situations.

Data Sources

There were two public data sources we used to cover the entirety of the 2025-26 NBA regular season. These sources were the official NBA statistics API and the Kalshi prediction market API. Data fetched from both was returned in JSON format, which we then converted to pandas DataFrames and eventually stored as CSV files.

Starting with NBA API, we used the LeagueGameLog endpoint (from the open-source nba_api python package) to query at the team level and return one record per team per game for all 1,230 regular-season games (resulting in 2460 total rows). Each of these rows contained column info with game ID, game date, team, and matchup, the game result (win or loss), and nineteen other box-score statistics (field goals, three-point field goals, and free throws made, attempted, and percentage; offensive, defensive, and total rebounds; assists; steals; blocks; turnovers; personal fouls; points; and plus-minus). We also used the BoxScoreTraditionalV3 endpoint to return team statistics accumulated from tip-off through the end of each quarter. Additionally, we used the PlayByPlayV3 endpoint, which supplied a log from each game of the exact pregame/quarter start times (as games typically actually start 5-15 mins after designated start times).

Next, we got our market data from the Kalshi prediction market website, which is a CFTC-regulated exchange whose public Trade API (v2) provides historical price data. For every NBA game, Kalshi lists two binary contracts (one for each team) that have a unique market ticker (for example, “KXNBAGAME-25OCT21GSWLAL-GSW”). We used the recorded game date and the two teams' abbreviations returned from the NBA API to custom build every individual market ticker from last season to match that specific format. Then, once this step was completed, we used the Kalshi historical candlestick endpoint to fetch all the one-minute candles in the five minutes before our four checkpoints (tip-off, start of the second, third, and fourth quarters) and kept only the most recent candle. The info for each row of this data included the opening, high, low, and closing bid/ask prices for the "yes" side of the contract, along with trading volume, open interest, and the candle's closing timestamp.

Overall, of the 9,840 possible market snapshots (1,230 games, two markets per game, and our four checkpoints) we were successfully able to retrieve 9,784 of them. The missing records were caused by three main issues. First, in our initial pull we found that there were five neutral-site games played (Mexico

City, Las Vegas, Berlin, and London) which meant there was no true home team and so we excluded all five of them (reducing our total to 1,225 games). Another issue was with the Golden State at Minnesota game played on January 25th (delayed by one day). Kalshi never updated the ticker (from the 24th) and because we built our tickers from scratch from the date the game was played, both of its markets returned "not found" at every checkpoint. Lastly, there was likely an interruption on Kalshi's side (markets were closed or paused temporarily) where no price data was returned for several games at different windows. The affected games included: Cleveland at Philadelphia (January 16, before tip-off), New Orleans at Indiana (January 16, before tip-off), Denver at Minnesota (November 15, before start of Q4), and Charlotte at Phoenix (March 8, before start of Q4). We decided that because of how few games were affected, they would be excluded rather than imputed.

Feature Engineering & Modeling Tables

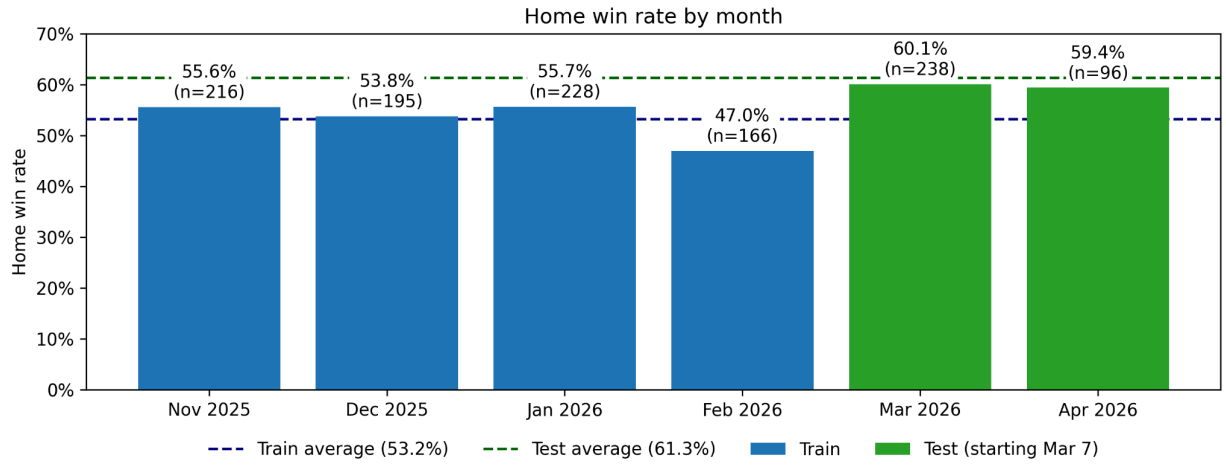
Since the goal of this project is to predict the results of NBA matchups, we built four files representing natural checkpoints in a game, one for pre-game data and three for in-game data at the end of the 1st, 2nd, and 3rd quarters. Each file has one row per game with HOME_WIN as the target variable. Each also includes KALSHI_PROB_HOME, the market's implied home-win probability, calculated by taking the bid/ask midpoint of each team's contract and dividing the home midpoint by the sum of both (since the two didn't always add up to exactly 1).

Next, specifically for pre-game file, we used 19 rolling features (ROLL_*) in which we took each team's average over its previous 5 games for each box stat. This worked well across the board except for shooting percentages as it needed to be recalculated as total makes divided by total attempts over the period window (so games with 40 shot attempts weren't being counted the same as games with 60). Additionally, in the first data pull for this file we noticed a major leakage issue caused by a bug in the code in which current games were included in the averages (so the previous five game stats plus the current game). This was easily fixed by just shifting each team's games back by one game and then re-pulling the data. Furthermore, for the in-game files, we used 18 in-game features (IG_*), in which we just took each team's cumulative box score up to the checkpoint, then added IG_SCORE_DIFF (the current score margin). Lastly, all rolling and in-game statistics were converted to a home-minus-away difference. The final pre-game table ended up with 20 features (the 19 ROLL_* plus KALSHI_PROB_HOME), while the final in-game tables each ended up with 38 features.

After the feature lists were set, we decided that games should be split chronologically rather than at random, so that no model learns from games played after the ones it's tested on. Requiring 5-game rolling averages for both teams removed 79 early-season games, and requiring Kalshi prices at all four checkpoints removed 5 more, leaving 1,141 games (Appendix Figure A1). Games before March 7 (854) were used for training and games from March 7 onward (287) for testing, and we tuned the hyperparameters with time-series cross-validation within the training set.

The split produced an uneven home win rate of 53.2% in training versus 61.3% in testing (Figure 1). By month, rates held steady between 53.8% and 55.7% from November through January, then fell to 47.0% in February, before rising to about 60% in March and April. February also had the fewest games of any training month (n=166), likely because of the All-Star break, so that dip rests on a smaller sample. The overall pattern is consistent with late-season tanking and teams resting players after clinching playoff spots, rather than a data error. It does matter for modeling though, since a model trained on a roughly 53% home win rate will likely underestimate home teams in the test period, which affects calibration and any comparison against the Kalshi probabilities.

Figure 1. Monthly home win rate across the 2025–26 season



Blue bars are training months and green bars are test months. Dashed lines show the overall train (53.2%) and test (61.3%) home win rates. March includes games from both sets, since the test split begins March 7.

Supervised Learning

Motivation

Prediction markets like Kalshi are often treated as efficient, meaning their prices should already reflect all publicly available information about a game. Our project tests that assumption for NBA games by asking whether box score statistics add predictive value beyond the Kalshi price at a given point in the game. If the market already prices in the box score, adding stats to a model should not improve on the market's own probability. If the stats do improve it, that shows where and when the market lags behind what is happening on the court.

We formulated this as a binary classification supervised learning problem, where the target label is whether the home team won (HOME_WIN) and the model outputs a predicted home-win probability. Because the information available changes as a game unfolds, we trained a separate model at each of our four checkpoints (pre-game, and the end of the 1st, 2nd, and 3rd quarters). We compared models with and without the Kalshi price, so we can measure what the stats add on top of the market rather than how they perform on their own.

Data Source

Please refer to the **Data Sources** and **Feature Engineering & Modeling Tables** sections above for details on the data sources and the features prepared for modeling. A full list of feature columns is provided in Appendix Table A1.

Methods Description

This section describes our modeling workflow, feature representations, model choices, and hyperparameter tuning.

Workflow

At each of the four checkpoints, we trained three models on three feature sets (Kalshi-only, NBA-only, and Combined), for 36 configurations in total. Games were split chronologically, with 854 games before March 7 for training and 287 games from March 7 onward for testing. Each model was evaluated with 5-fold time-series cross-validation on the training games and once on the test games. Each model was also compared against two baselines, a naive baseline that predicts the training home win rate for every game and the raw Kalshi price.

Feature Representations

All NBA statistics were represented as home-minus-away differences, so each feature measures one team's advantage over the other. Pre-game models use 5-game rolling averages, while in-game models add cumulative box score totals and the current score margin. The Kalshi-only models learn how the market price relates to actual outcomes, which could correct the market if it is consistently over- or under-confident. The feature sets and their sizes at each checkpoint can be found in Appendix Table A2.

Models

We chose three models from different families (Scikit-learn, n.d.). Logistic regression, a probabilistic linear model, is an interpretable baseline that produces well-calibrated probabilities. Random forest, a tree-based ensemble of 500 trees, captures non-linear effects and interactions, such as a favored team shooting poorly. K-nearest neighbors, an instance-based model, predicts from past games with a similar market price and score situation. Features were standardized for logistic regression and kNN so that larger-scale stats don't outweigh the Kalshi probability. Random forest was left unscaled, since tree splits don't depend on feature scale.

Hyperparameter Tuning

Each model was tuned with grid search using 5-fold time-series cross-validation (Scikit-learn, n.d.), where each fold trains on earlier games and validates on later ones. The hyperparameters tuned for each model were regularization strength for logistic regression, leaf size and features per split for random forest, and number of neighbors and weighting for k-nearest neighbors. The settings were then selected by Brier score, which rewards accurate probabilities and can be compared directly with the Kalshi price. The best settings were refit on all training games and scored once on the test set, and the grids were kept small to reduce the risk of overfitting the validation folds with only about 850 training games. Appendix Table A3 lists the values searched for each model.

Overall Results Reporting

We evaluated the models in three parts: overall results comparing each model family against the market, an in-depth analysis of the best models, and a failure analysis of specific games.

Evaluation Metrics

We used Brier score as our main metric, which is the average squared difference between the predicted home-win probability and the actual result (lower is better) (Scikit-learn, n.d.). It scores the probability itself rather than just whether the pick was right, so it rewards models that are both accurate and appropriately confident. It also works for the Kalshi price, which is already a probability, so we could score the market the same way and use it as a direct baseline.

We also tracked log loss and accuracy as secondary metrics (test-set results for all three are in Appendix Table A4). Log loss punishes confident wrong predictions more heavily (Scikit-learn, n.d.), which lets us check that our results don't depend on how errors are weighted. Accuracy is easy to understand but ignores confidence, and it can be misleading on our test set. The home team won 61.3% of test games, so simply picking the home team every time scores an accuracy of 0.613.

We checked test-set differences with a paired bootstrap (2,000 resamples) and treated a difference as significant when its 95% confidence interval did not include zero.

Model Comparison

Table 1 compares the best version of each model family against the Kalshi market at each checkpoint, using the cross-validated Brier score.

Table 1. Best model from each family vs. Kalshi market, CV mean ± std, by checkpoint

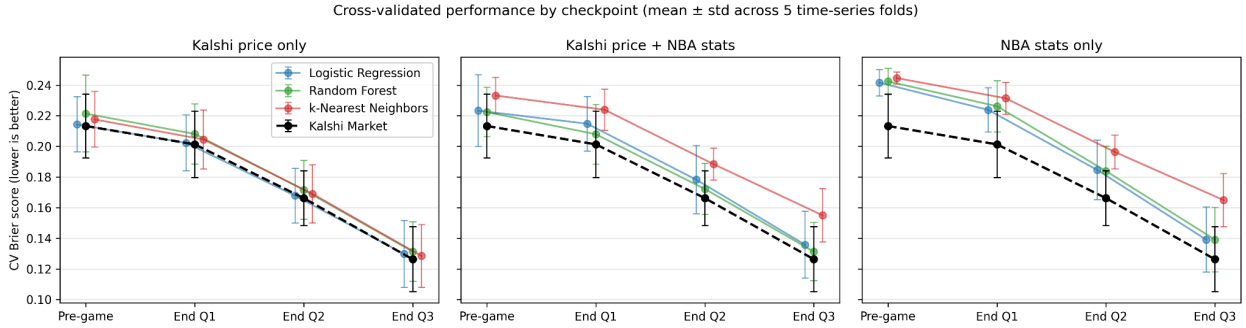
Checkpoint	Logistic Regression	Random Forest	kNN	Kalshi Market
Pre-Game	0.214 ± 0.018 (K)	0.221 ± 0.025 (K)	0.218 ± 0.018 (K)	0.213 ± 0.021
End Q1	0.202 ± 0.018 (K)	0.208 ± 0.020 (C)	0.205 ± 0.019 (K)	0.201 ± 0.022
End Q2	0.168 ± 0.018 (K)	0.172 ± 0.019 (K)	0.169 ± 0.019 (K)	0.166 ± 0.018
End Q3	0.130 ± 0.022 (K)	0.131 ± 0.019 (C)	0.129 ± 0.021 (K)	0.126 ± 0.021

K = Kalshi-only, C = Combined. Lower is better.

The Kalshi market had the lowest cross-validated Brier score at every checkpoint. The closest models were only 0.001 to 0.003 behind, which is well within the fold-to-fold standard deviation of about 0.02. The best version of each model family almost always used only the Kalshi price.

The test set showed the same pattern (Appendix Table A4). The market had the lowest log loss at every checkpoint and the lowest Brier score at all but End Q3, where the Kalshi-only logistic regression beat it by just 0.001. A few models scored slightly higher accuracy than the market, but the largest gap (combined random forest at End Q3, 0.913 vs. 0.899) was only four games, and that model's log loss was worse than the market's. Figure 2 breaks the cross-validated results out by feature set, showing how each model family performed with the Kalshi price only, with Kalshi and NBA stats combined, and with NBA stats only.

Figure 2. Cross-validated performance by checkpoint

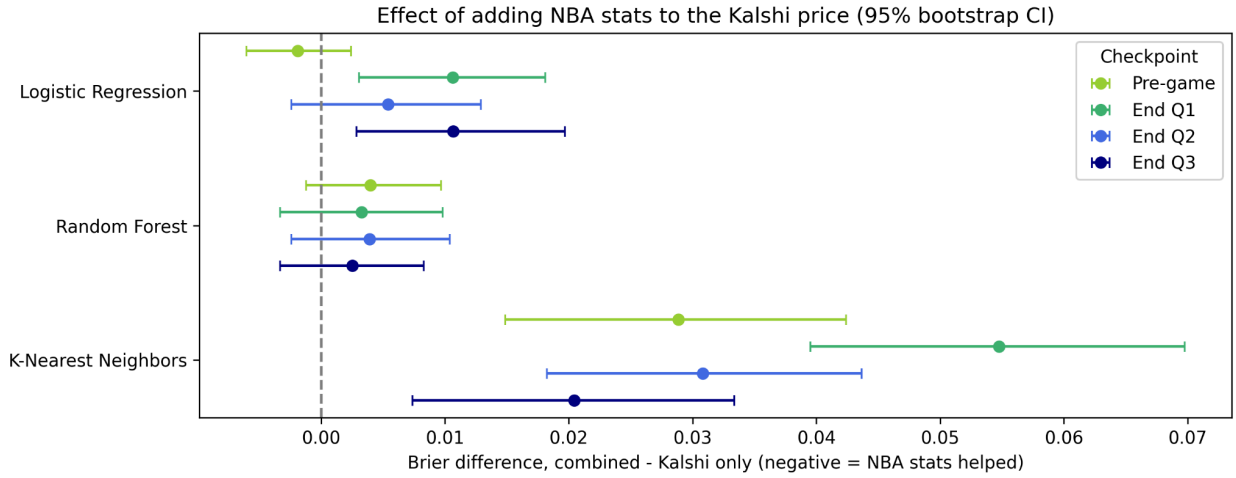


With the Kalshi price alone, all three models land almost exactly on the market line. Adding NBA stats pushes every model above it, with kNN falling furthest behind, since the many weak rolling stats dilute the market price. NBA stats alone start about 0.03 behind the market pre-game but close to about 0.013 by the end of Q3.

Do NBA statistics add value beyond the market?

To answer our main question directly, we compared each model's combined version against its Kalshi-only version on the test set. Figure 3 shows the difference in Brier score (combined minus Kalshi-only) for each model and checkpoint, with 95% confidence intervals from a paired bootstrap. A point left of zero means the NBA stats helped, a point right of zero means they hurt, and an interval that crosses zero means there was no significant difference.

Figure 3. Effect of adding NBA stats to the Kalshi price



Adding NBA stats to the Kalshi price never significantly improved test-set predictions (0 of 12 comparisons). In 6 of the 12, it made them significantly worse, including kNN at every checkpoint and logistic regression at End Q1 and End Q3. Random forest showed no significant difference at any checkpoint. This suggests the market price already reflects the score and each team's recent performance.

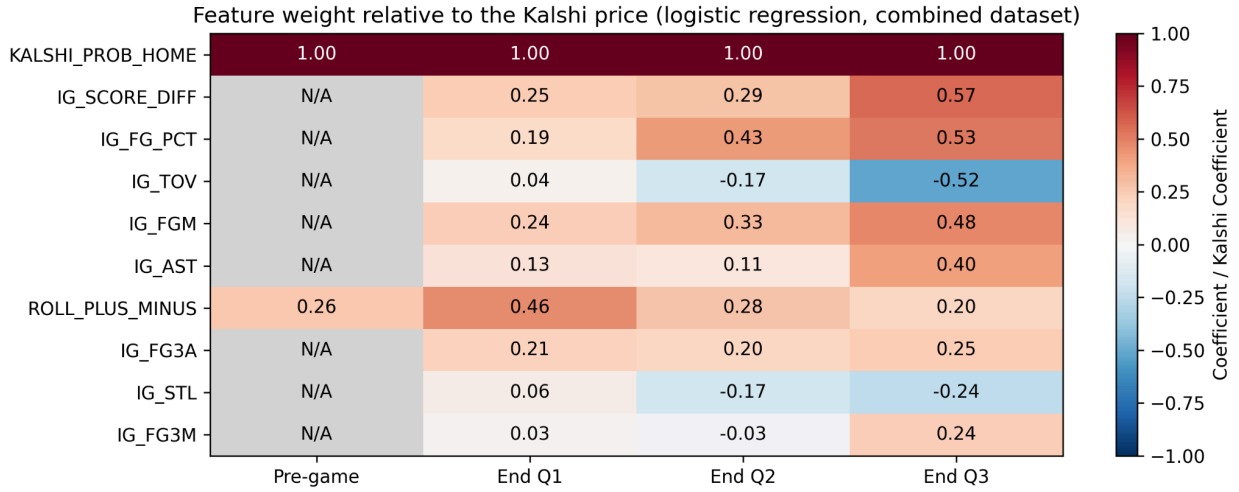
In-Depth Evaluation

The best-performing models used only the Kalshi price, which leaves a single feature and little to analyze. These models mostly learned to repeat the market. Their cross-validated Brier scores were within 0.001 to 0.003 of the market's at every checkpoint (Table 1), which suggests the market price was already well calibrated and left little for a model to correct. Figure 3 showed that adding NBA stats never improved test-set predictions, and this section looks at why. It focuses on combined logistic regression, the strongest model that uses NBA stats. Its coefficients are easy to interpret, and the ablation analysis below starts from the Kalshi-only model and adds feature groups to it one at a time.

Feature Importance

To see which features the model relied on, we compared each feature's coefficient to the Kalshi coefficient at the same checkpoint. Figure 4 shows the 10 features with the largest weights, scaled so the Kalshi price equals 1.00 in every column.

Figure 4. Feature weight relative to the Kalshi price



The market price carried the most weight at every checkpoint. In-game stats gained weight as the game went on, and by the end of Q3 the score margin (0.57), field-goal percentage (0.53), turnovers (−0.52), and field goals made (0.48) each carried about half the weight of the market price. ROLL_PLUS_MINUS was the only rolling stat in the top 10. Several box-score stats overlap (for example, field goals made, field-goal percentage, and the score margin), so some signs, like the negative coefficient on steals, shouldn't be read individually. Even though the model leaned on in-game stats during training, they didn't improve test-set predictions, so whatever they captured was likely already in the market price.

Ablation Analysis

To see which feature groups helped or hurt, we started from the Kalshi-only model (our best model) and added one group of NBA features at a time using logistic regression. Table 2 shows the cross-validated Brier score for each combination.

Table 2. CV Brier by feature group added to the Kalshi price (logistic regression)

Checkpoint	Pre-Game	End Q1	End Q2	End Q3
Kalshi Only	0.214 ± 0.018	0.202 ± 0.018	0.168 ± 0.018	0.130 ± 0.022
+ Score Margin	–	0.203 ± 0.018	0.166 ± 0.015	0.129 ± 0.021
+ In-Game Stats	–	0.210 ± 0.021	0.169 ± 0.017	0.129 ± 0.019
+ Rolling Stats	0.223 ± 0.024	0.211 ± 0.024	0.178 ± 0.025	0.138 ± 0.023
+ All NBA Stats	0.223 ± 0.024	0.215 ± 0.018	0.178 ± 0.022	0.136 ± 0.022

Each row adds one feature group to the Kalshi price on its own, so rows are not cumulative. In-game features don't exist pre-game. Lower is better.

Adding the score margin barely changed the Brier score, and the full in-game box score hurt at End Q1 but was roughly neutral by End Q3. The rolling stats did the most damage, worsening the score by about 0.009 at every checkpoint. That change is smaller than the fold-to-fold standard deviation, but it showed

up consistently at all four checkpoints. Rolling stats describe each team's recent form, which the market price already accounts for, so they mostly gave the model extra noise to fit.

Sensitivity Analysis

To check whether our results depended on tuning choices, we compared the best and worst cross-validated Brier scores across each model's full hyperparameter grid. We used the combined feature set, since the Kalshi-only models have a single feature and little to tune. Appendix Table A5 shows the range for each model at each checkpoint.

Random forest was the most sensitive to its settings. Its End Q3 score ranged from 0.131 to 0.194 depending on leaf size, a spread larger than the gap between any model and the market. Logistic regression was the most stable, and almost all of its range came from the most heavily regularized setting ($C = 0.001$). With C from 0.01 to 1, its End Q3 range narrowed to 0.136–0.143. kNN varied the least but was never competitive once NBA stats were included. Sensitivity generally grew as the game went on, especially for the random forest models. Most importantly, the best setting of every model was still worse than the market at every checkpoint, so different tuning wouldn't change our main finding.

Tradeoffs

The clearest tradeoff is between interpretability and flexibility. Logistic regression is easy to interpret and is stable across settings, while random forest can capture more complex patterns but depends heavily on tuning (Appendix Table A5). That flexibility sometimes paid off, as random forest beat logistic regression on the test set at End Q3 (0.082 vs. 0.087 Brier). It also came at a cost to calibration as the combined random forest model had the highest End Q3 test accuracy (0.913 vs. 0.899 for the market), but its log loss was worse (0.275 vs. 0.253), meaning it picked more winners while being less reliable about how confident to be (Appendix Table A4).

A second tradeoff is between adding information and adding noise. More NBA stats gave the models more to work with, but with only about 850 training games, features that overlapped with the market price mostly made predictions worse. Timing creates a similar tension, since predictions become much more accurate later in the game (market Brier of 0.213 pre-game vs. 0.126 at End Q3), but late-game outcomes are also easier to call, which leaves less room for any model to beat the market.

Failure Analysis

To find where the model went wrong, we looked at test games at the end of Q3, when predictions are most confident, and selected the biggest failure of combined logistic regression in each of three categories. The effect columns in Table 3 show how much each feature group pushed the prediction toward a home win (positive) or an away win (negative), in log-odds.

Table 3. Failure examples at End Q3 (combined logistic regression)

Category	Game	Score	Home Lead	Market Prob	Model Prob	Kalshi Effect	Rolling Effect	In-game Effect
Random Error	BKN @ DET, 3/7	105 - 107 (BKN won)	+11	97.5%	95.0%	+1.04	+0.27	+1.38
Noisy Features	PHX @ ORL, 3/31	115 - 111 (ORL won)	0	56.5%	24.2%	+0.08	-0.63	-0.86
Missing Info	TOR @ DEN, 3/20	121 - 115 (DEN won)	-9	48.5%	21.8%	-0.10	-0.28	-1.16

The three failures show different causes. In the random error, Detroit led Brooklyn by 11 after Q3, and both the market (97.5%) and the model (95.0%) favored Detroit before Brooklyn won the fourth quarter by 13. Nothing in the data pointed to that collapse, so no model could have predicted it. In the noisy features case, Phoenix and Orlando were tied, and the market's 56.5% for Orlando was barely enough to move the model (a Kalshi effect of +0.08), so the rolling (-0.63) and in-game (-0.86) stats pulled it down to 24.2% before Orlando won. In the missing information case, Denver trailed Toronto by 9, yet the market still gave them 48.5%, suggesting it knew something the box score didn't. The model treated the deficit as new information even though the market price already reflected it (an in-game effect of -1.16), and dropped to 21.8% before Denver won 121–115.

Several future improvements could address these failures, though random errors like the Detroit game can't be fixed with better features. Dropping the rolling stats or regularizing the NBA features more heavily would keep weak signals from deciding close games. Using the Kalshi price as a fixed offset, so the NBA features only adjust it, would stop the model from counting the score twice. Adding team-strength and injury features could capture what the market may have known in the Denver game.

Unsupervised Learning

Methods Description

Two clustering methods were explored: K-means and agglomerative hierarchical clustering. K-means partitioned the observations by minimizing within cluster squared distances. It was used due to the ability to identify broad patterns in numerical data (Müller & Guido, 2016). Hierarchical clustering builds groups by progressively merging clusters, providing a different perspective on the underlying structure and a comparison for K-means (Müller & Guido, 2016). Both methods were applied to standardized numerical features at each checkpoint.

The feature representation included rolling basketball statistics and, for in game checkpoints, statistics accumulated during the current game. Features included shooting, rebounds, assists, steals, blocks, turnovers, fouls, points, plus-minus, and current score differential. Two feature variants were compared: one including Kalshi's home-win probability and one excluding it. This comparison assessed whether clusters reflected basketball performance independently of market probabilities.

Hyperparameter exploration evaluated K-means with two through eight clusters, using 20 initializations and a fixed random seed. The silhouette score was used as the primary selection criterion, supported by inertia, the Calinski–Harabasz score, and the Davies–Bouldin score (Scikit-learn, n.d.). Agglomerative clustering used Ward linkage and the same selected cluster count to facilitate comparison (Scikit-learn, n.d.). Principal component analysis (PCA) projected the standardized features into two dimensions for visualization.

Evaluation & Results

Model quality was assessed using complementary metrics. The silhouette score measured the cluster separation, the Calinski–Harabasz score compared the between cluster and within cluster separation, and the Davies–Bouldin score evaluated cluster similarity (Scikit-learn, n.d.). Inertia measured within cluster squared distances, while the adjusted Rand index (ARI) measured agreement between the two clustering methods (Scikit-learn, n.d.).

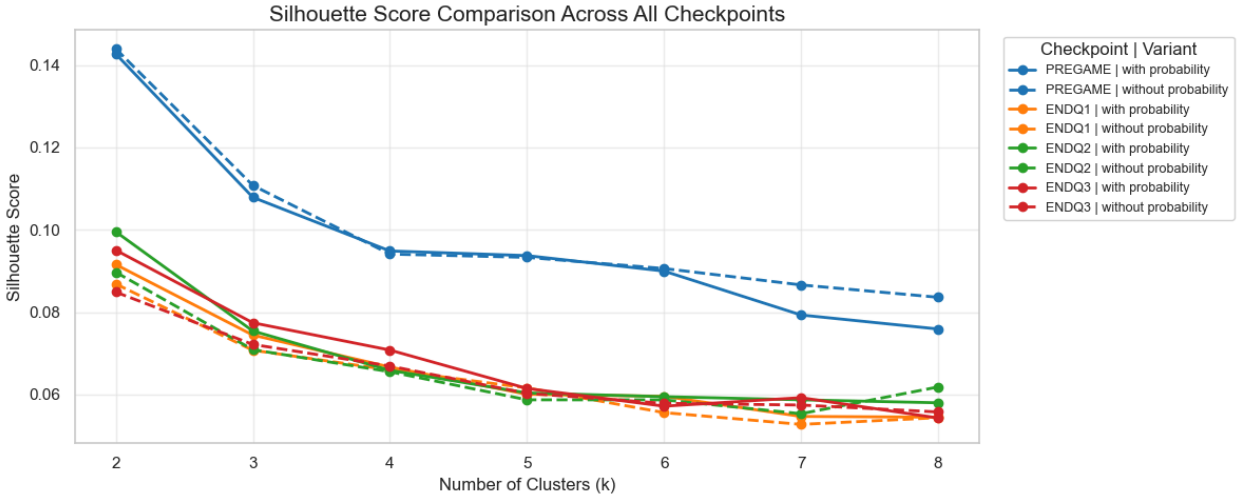
Table 4. Summary Table. K-means had higher silhouette scores across all checkpoints compared to hierarchical clustering.

K-MEANS VS. HIERARCHICAL

	checkpoint	variant	selected_cluster_count	kmeans_silhouette_score	hierarchical_silhouette_score	adjusted_rand_index
0	pregame	with_probability	2	0.1427	0.1192	0.1997
1	pregame	without_probability	2	0.1439	0.1172	0.4561
2	endq1	with_probability	2	0.0915	0.0555	0.2143
3	endq1	without_probability	2	0.0868	0.0485	0.2128
4	endq2	with_probability	2	0.0995	0.0731	0.3726
5	endq2	without_probability	2	0.0897	0.0559	0.2622
6	endq3	with_probability	2	0.0950	0.0734	0.4948
7	endq3	without_probability	2	0.0849	0.0575	0.2307

K-means selected two clusters for all checkpoint and feature-variant combinations. Pregame silhouette scores were approximately 0.143–0.144, compared with 0.085–0.100 at the in-game checkpoints. These relatively low scores indicate overlapping game-state patterns rather than sharply separated groups. Cluster profiles showed interpretable differences in team performance and game state. By ENDQ3, the clusters had substantially different average standardized score-differential features, approximately −10.44 and +12.13 in the variant including Kalshi probability.

Figure 5. Silhouette score K means clustering comparison across all checkpoints. The silhouette scores are highest at k=2 across all checkpoints and both feature variants.



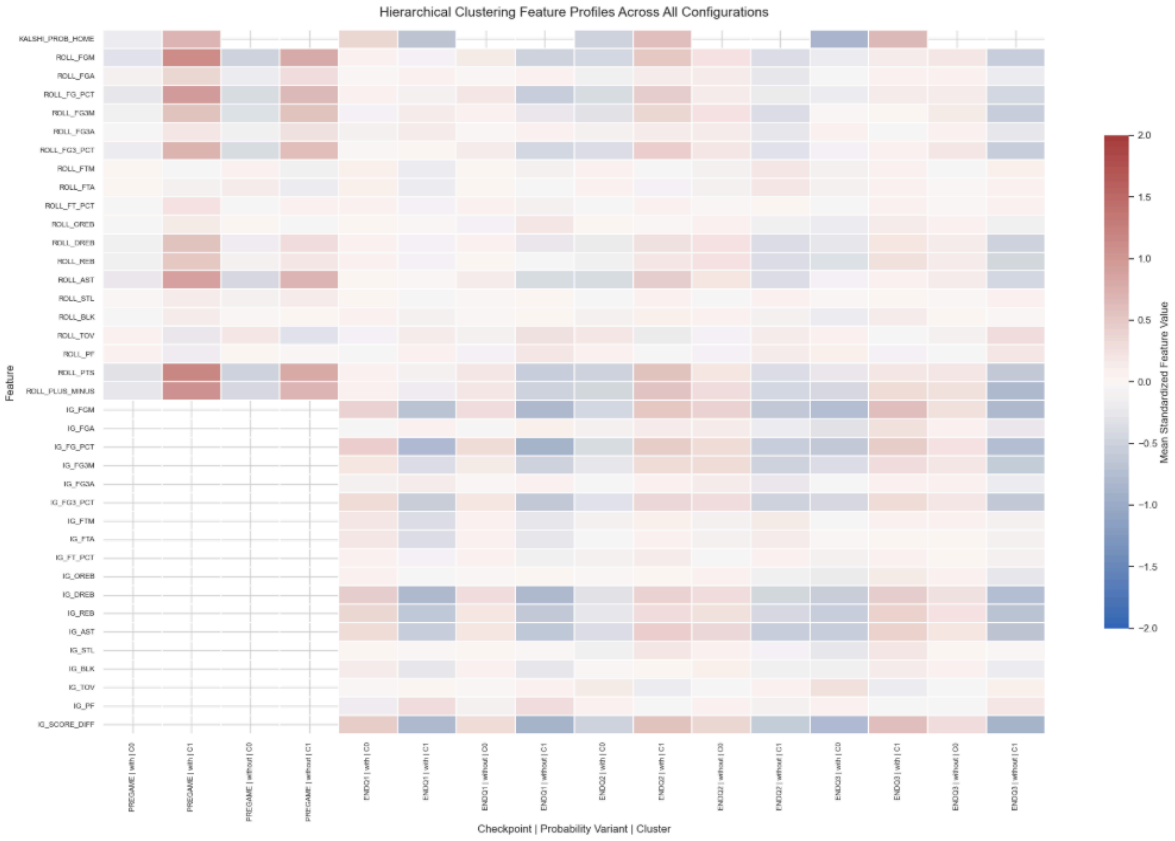
Hierarchical clustering produced lower silhouette scores, ranging from approximately 0.048 to 0.119. Agreement with K-means was mixed, with ARI values between approximately 0.20 and 0.50. The highest agreement occurred at ENDQ3 when Kalshi probability was included (ARI ≈ 0.495). The PCA visualizations below show the results by illustrating the structure of the hierarchical clusters across checkpoints and feature variants.

Figure 6. Hierarchical Clustering PCA Scatterplots



The scatterplots display game observations projected onto the first two principal components, with colors representing cluster assignments from agglomerative hierarchical clustering. Each panel corresponds to a prediction checkpoint and a feature variant, either including or excluding Kalshi's implied home-win probability.

Figure 7. Hierarchical Clustering Feature Profiles



This heatmap shows standardized feature values across hierarchical clusters, checkpoints, and feature variants. Values above zero (red) indicate that a cluster's mean for a feature is above the overall standardized mean, while values below zero (blue) indicate a below average value.

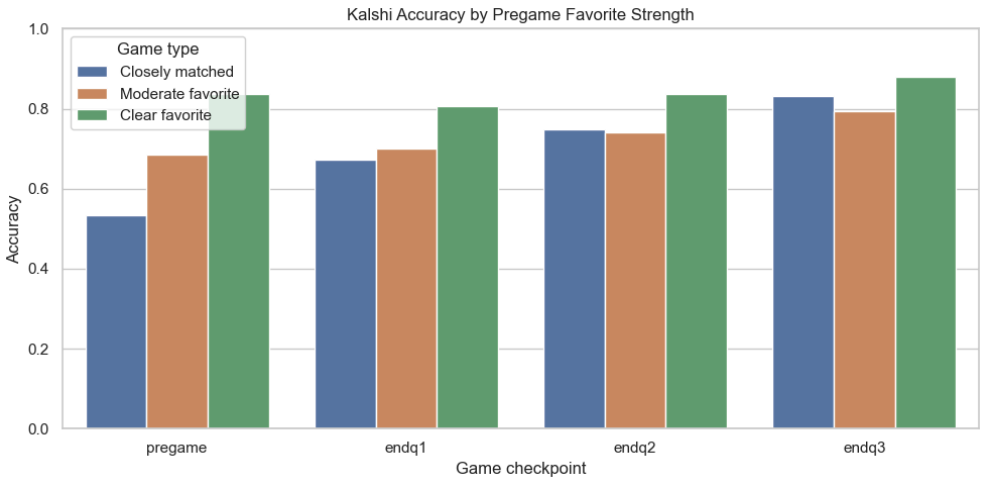
The differences between hierarchical clustering and K-means, as reflected in the ARI values, also suggest that the resulting groupings depend partly on the clustering algorithm. Overall, hierarchical clustering provides a useful comparison to K-means, but its results offer limited evidence of strongly separated game categories.

Next, Kalshi's performance was evaluated using accuracy, Brier score, log loss, and calibration gap. Accuracy measured the proportion of correctly classified outcomes using a 0.50 probability threshold (Scikit-learn, n.d.). The Brier score measured squared probability errors, and log loss penalized incorrect predictions more heavily when made with high confidence (Scikit-learn, n.d.). Calibration gap was calculated as the observed home-win rate minus the mean predicted home-win probability, with values closer to zero indicating better aggregate calibration (Scikit-learn, n.d.).

Weighted accuracy increased from 69.9% pregame to 73.2% at ENDQ1, 77.7% at ENDQ2, and 83.5% at ENDQ3. Meanwhile, the Brier score decreased from 0.1923 to 0.1130, and log loss decreased from 0.5648 to 0.3485. Together, these results show that Kalshi's forecasts became more accurate as additional in-game information became available.

To examine performance across game types, games were grouped by pregame favorite strength using the absolute difference between Kalshi's home-win probability and 0.50. The groups consisted of closely matched games, moderate favorites, and clear favorites. Clear-favorite games achieved 83.66% pregame accuracy, compared with 53.40% for closely matched games. By ENDQ3, accuracy increased to 88.05% for clear favorites and 83.17% for closely matched games. This showed that clear-favorite games were easier to predict before tipoff, while the accuracy gap narrowed as the game progressed.

Figure 8. Kalshi Accuracy by Pregame Favorite Strength



Accuracy increased across all three favorite-strength groups as games progressed, with the largest improvement among closely matched games, from approximately 53% pregame to 83% by the end of the third quarter.

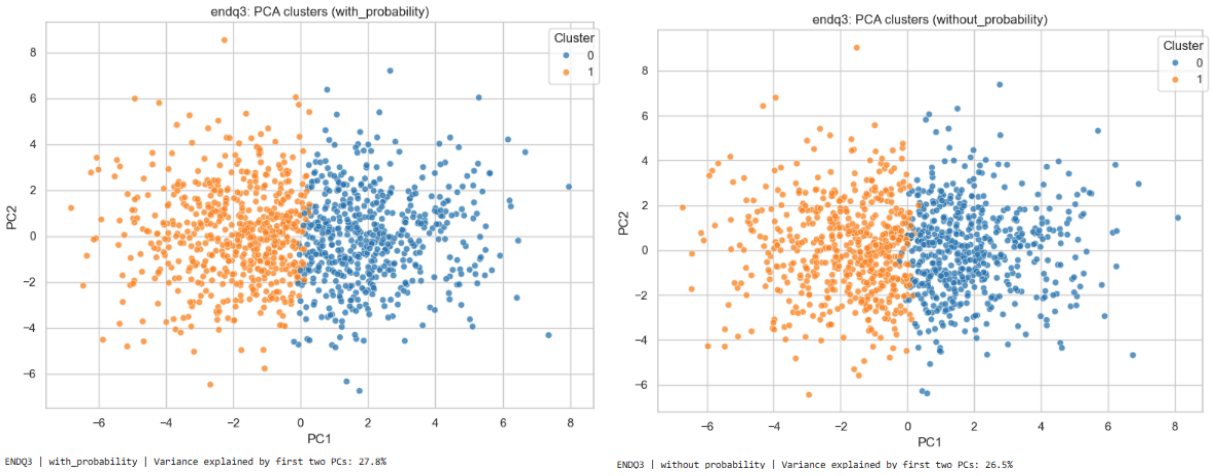
Probability movement was also associated with performance. Movement was measured as the absolute difference between the current checkpoint probability and the pregame probability. At ENDQ3, high-movement games achieved 86.0% accuracy and a Brier score of 0.0993, compared with 74.8% accuracy and a Brier score of 0.1664 for low-movement games. This association showed that games with larger changes in market probabilities were more predictable by the end of the third quarter, but it does not show causation.

Sensitivity Analysis

Sensitivity was assessed by varying the number of K-means clusters from two to eight and comparing feature sets with and without Kalshi probability. Silhouette scores were highest at two clusters for every checkpoint and feature variant, while inertia decreased as more clusters were added. This is expected because additional clusters can reduce within-cluster variation, but the declining silhouette scores suggest that additional clusters did not improve separation.

The feature comparison showed similar accuracy, Brier score, and log loss, despite some differences in cluster assignments and profiles. This shows the added probability did not materially improve the evaluation. However, PCA captured only approximately 25.6%–38.6% of total variance.

Figure 9. K-Means END-Q3 PCA



The scatterplots show that K-Means identified 2 clusters at ENDQ3, with the most separation occurring along the first principal component. Including the Kalshi probability (left, 27.8% of variance explained) and without (right, 26.5% of the variance explained) shows a similar cluster structure between the two, but the plots only capture part of the overall feature structure.

Discussion

Supervised Learning

The most surprising result was how completely the market dominated. We expected in-game stats to catch moments when the Kalshi price lagged behind the action, but adding NBA stats never improved test-set predictions in any of the 12 comparisons. Even the Kalshi-only models, which could have corrected a miscalibrated market, landed almost exactly on the market's own Brier score.

Our biggest challenge was working with a single season of data. With only about 850 training games, we kept the hyperparameter grids small and tested differences with a paired bootstrap to limit overfitting. The chronological split also left training with a lower home win rate than testing (53.2% vs. 61.3%), but we kept it, since a random split would let models learn from future games.

With more time, we would collect several seasons of data to test whether the results hold over time. We would also try starting each prediction from the market price and letting the NBA stats only adjust it, add team-strength and injury features, and check prices more often than once per quarter to see whether the market briefly lags behind the game.

Unsupervised Learning

One surprising finding was that K-means consistently selected two clusters across all four checkpoints and both feature variants. The initial expectation was the games would separate into a wider range of groups based on team performance and market characteristics. However, the relatively low silhouette scores suggested that the games did not form sharply separated groups, even when two clusters were preferred.

A key challenge was separating the quality of the unsupervised representations from the predictive performance of Kalshi. We addressed this by evaluating clustering quality with internal metrics and assessing Kalshi predictions separately using probability-based metrics. Comparing feature sets with and without Kalshi probability also helped us investigate whether the market feature substantially changed the resulting groupings or their associated predictive summaries.

With additional time, we would evaluate cluster stability through resampling, examine alternative feature subsets, and report uncertainty intervals and subgroup sample sizes. We could also look at comparisons to other sports leagues, such as the MLB, NFL, or NHL and investigate both the similarities and differences between leagues and Kalshi predictions.

Ethical Considerations

For supervised learning, since Kalshi data is for prediction markets, the findings could encourage gambling if users interpret model performance as a guarantee of future outcomes. We would address this ethical issue by stating the aim was to evaluate predictive accuracy rather than recommending bets, and by acknowledging the limitations in the generalizability of the data since it was only 1 year.

For unsupervised learning, bias could arise if the results are generalized beyond the limited sample of NBA games with the available Kalshi data. The clusters could also be read as distinct game types, even though they were only weakly separated. To address this, we would avoid claiming that clustering causes better predictions or that historical accuracy from last season guarantees future performance.

Statement of Work

Bradley led data collection and wrote the corresponding sections of the report, Ryan led the supervised learning analysis and its report sections, and Chris led the unsupervised learning analysis and its report sections. Each team member also supported the others on their portions of the project.

References

Angelini, G., & De Angelis, L. (2026). *When do markets fully process public information? Evidence from real-time prediction markets*. https://doi.org/10.48550/arXiv.2606.07811

Kalshi (n.d.). Get historical market candlesticks. https://docs.kalshi.com/api-reference/historical/get-historical-market-candlesticks

Kalshi (n.d.). Historical data. https://docs.kalshi.com/getting_started/historical_data

Kalshi (n.d.). Rate limits and tiers. https://docs.kalshi.com/getting_started/rate_limits

Maddox, J. T., Sides, R., & Harvill, J. L. (2022). *Bayesian estimation of in-game home team win probability for National Basketball Association games*. https://doi.org/10.48550/arXiv.2207.05114

Müller, A. C., & Guido, S. (2016). *Introduction to machine learning with Python: A guide for data scientists*. O'Reilly Media.

Patel, S. (n.d.). BoxScoreTraditionalV3. nba_api documentation. https://github.com/swar/nba_api/blob/master/docs/nba_api/stats/endpoints/boxscoretraditionalv3.md

Patel, S. (n.d.). LeagueGameLog. nba_api documentation. https://github.com/swar/nba_api/blob/master/docs/nba_api/stats/endpoints/leaguegamelog.md

Patel, S. (n.d.). nba_api [Computer software]. GitHub. https://github.com/swar/nba_api

Patel, S. (n.d.). PlayByPlayV3. nba_api documentation. https://github.com/swar/nba_api/blob/master/docs/nba_api/stats/endpoints/playbyplayv3.md

Scikit-learn (n.d.). Accuracy_score. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

Scikit-learn (n.d.). Brier_score_loss. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.brier_score_loss.html

Scikit-learn (n.d.). *Clustering performance evaluation*. https://scikit-learn.org/stable/modules/clustering.html#clustering-performance-evaluation

Scikit-learn (n.d.). GridSearchCV. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

Scikit-learn (n.d.). *Hierarchical clustering*. https://scikit-learn.org/stable/modules/clustering.html#hierarchical-clustering

Scikit-learn (n.d.). *K-Means*. https://scikit-learn.org/stable/modules/clustering.html#k-means

Scikit-learn (n.d.). KNeighborsClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

Scikit-learn (n.d.). log_loss. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.log_loss.html

Scikit-learn (n.d.). LogisticRegression. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

Scikit-learn (n.d.). Probability calibration. https://scikit-learn.org/stable/modules/calibration.html

Scikit-learn (n.d.). RandomForestClassifier. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

Scikit-learn (n.d.). TimeSeriesSplit. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html

Spann, M., & Skiera, B. (2009). Sports forecasting: A comparison of the forecast accuracy of prediction markets, betting odds and tipsters. *Journal of Forecasting, 28*(1), 55–72. https://doi.org/10.1002/for.1091

Wolfers, J., & Zitzewitz, E. (2004). Prediction markets. *Journal of Economic Perspectives, 18*(2), 107–126. https://doi.org/10.1257/0895330041371321

Appendix

Figure A1. Games Remaining at Each Data Prep Step

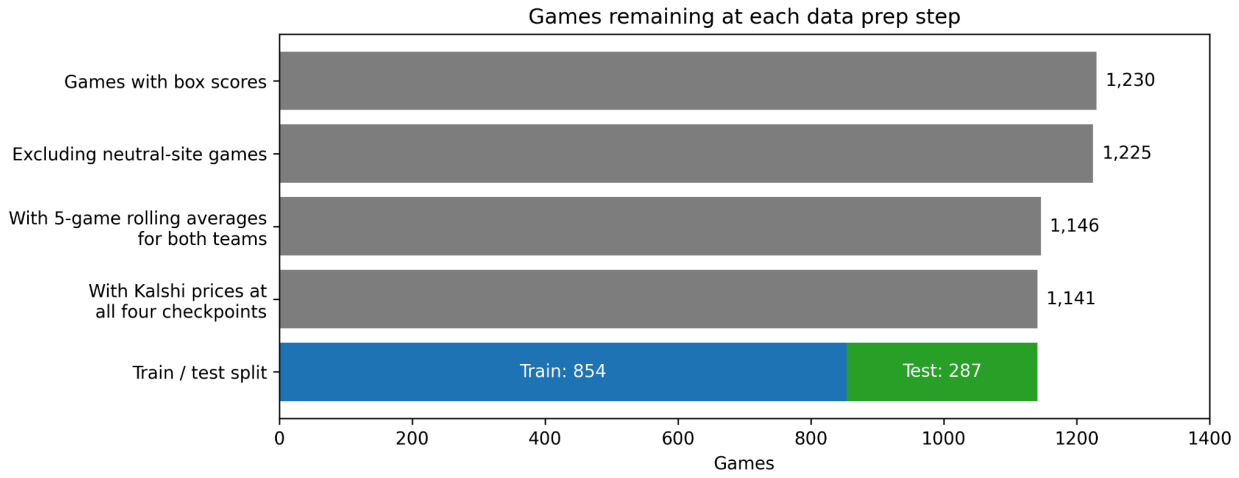


Table A1. Feature List

Feature	Description	Data Type	Usage
GAME_ID	Unique NBA game identifier, used to join the NBA and Kalshi data	int	All
GAME_DATE	Date the game was played, used to sort games and set the chronological train/test split	datetime	All
HOME_WIN	Target variable: 1 if the home team won, 0 if the away team won	int	All
KALSHI_PROB_HOME	Kalshi market's implied home-win probability at the checkpoint: the home contract's bid/ask midpoint divided by the sum of the home and away midpoints (0 to 1)	float	All
ROLL_FGM	Difference in average field goals made per game	float	All
ROLL_FGA	Difference in average field goals attempted per game	float	All
ROLL_FG_PCT	Difference in field goal percentage (total makes / total attempts over the window)	float	All
ROLL_FG3M	Difference in average three-point field goals made per game	float	All
ROLL_FG3A	Difference in average three-point field goals attempted per game	float	All
ROLL_FG3_PCT	Difference in three-point percentage (total	float	All

	makes / total attempts over the window)		
ROLL_FTM	Difference in average free throws made per game	float	All
ROLL_FTA	Difference in average free throws attempted per game	float	All
ROLL_FT_PCT	Difference in free throw percentage (total makes / total attempts over the window)	float	All
ROLL_OREB	Difference in average offensive rebounds per game	float	All
ROLL_DREB	Difference in average defensive rebounds per game	float	All
ROLL_REB	Difference in average total rebounds per game	float	All
ROLL_AST	Difference in average assists per game	float	All
ROLL_STL	Difference in average steals per game	float	All
ROLL_BLK	Difference in average blocks per game	float	All
ROLL_TOV	Difference in average turnovers per game	float	All
ROLL_PF	Difference in average personal fouls per game	float	All
ROLL_PTS	Difference in average points scored per game	float	All
ROLL_PLUS_MINUS	Difference in average point margin (plus-minus) per game	float	All
IG_FGM	Difference in field goals made so far	int	Q1-Q3
IG_FGA	Difference in field goals attempted so far	int	Q1-Q3
IG_FG_PCT	Difference in field goal percentage so far	float	Q1-Q3
IG_FG3M	Difference in three-point field goals made so far	int	Q1-Q3
IG_FG3A	Difference in three-point field goals attempted so far	int	Q1-Q3
IG_FG3_PCT	Difference in three-point percentage so far	float	Q1-Q3
IG_FTM	Difference in free throws made so far	int	Q1-Q3
IG_FTA	Difference in free throws attempted so far	int	Q1-Q3
IG_FT_PCT	Difference in free throw percentage so far	float	Q1-Q3
IG_OREB	Difference in offensive rebounds so far	int	Q1-Q3

IG_DREB	Difference in defensive rebounds so far	int	Q1-Q3
IG_REB	Difference in total rebounds so far	int	Q1-Q3
IG_AST	Difference in assists so far	int	Q1-Q3
IG_STL	Difference in steals so far	int	Q1-Q3
IG_BLK	Difference in blocks so far	int	Q1-Q3
IG_TOV	Difference in turnovers so far	int	Q1-Q3
IG_PF	Difference in personal fouls so far	int	Q1-Q3
IG_SCORE_DIFF	Current score margin (home points minus away points), taken directly from the home team's plus-minus	float	Q1-Q3
SPLIT	Chronological split label: "train" for games before March 7, 2026, "test" from March 7 onward (not a model input)	object	All

Table A2. Feature sets by checkpoint

Feature Set	Pre-Game	In-Game (Q1-Q3)
Kalshi-Only	KALSHI_PROB_HOME (1)	KALSHI_PROB_HOME (1)
NBA-Only	19 ROLL_* (19)	19 ROLL_* + 18 IG_* (37)
Combined	Kalshi + NBA (20)	Kalshi + NBA (38)

Table A3. Hyperparameter search spaces

Model	Hyperparameter	Values searched
Logistic Regression	C (regularization strength; smaller = stronger)	0.001, 0.01, 0.1, 1
Random Forest	min_samples_leaf (minimum games per leaf)	25, 50, 100
	max_features (share of features per split)	sqrt, 0.5
K-Nearest Neighbors	n_neighbors (number of similar games)	25, 50, 75, 100
	weights	uniform, distance

Table A4. Test-Set Results by Feature Set, Model, and Checkpoint

Panel A: Brier score (lower is better)

Feature Set	Model	Pre-Game	End Q1	End Q2	End Q3

Baseline	Kalshi Market	**0.148**	**0.130**	**0.106**	0.077
	Naive	0.244	0.244	0.244	0.244
Kalshi-Only	Logistic Regression	0.153	0.135	0.109	**0.076**
	Random Forest	0.160	0.138	0.113	0.080
	kNN	0.156	0.140	0.109	0.085
NBA-Only	Logistic Regression	0.203	0.166	0.127	0.097
	Random Forest	0.205	0.172	0.134	0.097
	kNN	0.219	0.204	0.153	0.118
Combined	Logistic Regression	0.151	0.146	0.114	0.087
	Random Forest	0.164	0.141	0.117	0.082
	kNN	0.184	0.194	0.140	0.105

Panel B: Log loss (lower is better)

Feature Set	Model	Pre-Game	End Q1	End Q2	End Q3
Baseline	Kalshi Market	**0.458**	**0.412**	**0.341**	**0.253**
	Naive	0.681	0.681	0.681	0.681
Kalshi-Only	Logistic Regression	0.478	0.436	0.356	0.263
	Random Forest	0.498	0.436	0.361	**0.253**
	kNN	0.483	0.442	0.348	0.263
NBA-Only	Logistic Regression	0.596	0.511	0.394	0.309
	Random Forest	0.599	0.526	0.426	0.320
	kNN	0.630	0.597	0.480	0.385
Combined	Logistic Regression	0.471	0.463	0.361	0.285
	Random Forest	0.506	0.452	0.376	0.275
	kNN	0.554	0.575	0.443	0.350

Panel C: Accuracy (higher is better)

Feature Set	Model	Pre-Game	End Q1	End Q2	End Q3
Baseline	Kalshi Market	0.788	**0.843**	0.833	0.899

	Naive	0.613	0.613	0.613	0.613
Kalshi-Only	Logistic Regression	**0.791**	0.836	**0.836**	0.896
	Random Forest	0.784	0.833	0.822	0.889
	kNN	0.760	0.836	0.826	0.871
NBA-Only	Logistic Regression	0.725	0.788	0.815	0.878
	Random Forest	0.707	0.760	0.805	0.882
	kNN	0.659	0.704	0.801	0.840
Combined	Logistic Regression	0.788	0.822	0.826	0.892
	Random Forest	0.767	0.826	0.826	**0.913**
	kNN	0.753	0.714	0.784	0.857

All values are from a single evaluation on the 287 held-out test games (March 7, 2026 onward). Bold/highlighted cells mark the best value in each column; ties are both marked. Cross-validated mean ± std for model comparison is reported in Table 1.

Table A5. Sensitivity of CV Brier Score to Hyperparameters (combined feature set)

Model	Pre-Game	End Q1	End Q2	End Q3	Most influential setting
Logistic Regression	0.223 - 0.241	0.215 - 0.230	0.178 - 0.208	0.136 - 0.188	C (only C = 0.001 clearly hurts)
Random Forest	0.222 - 0.238	0.208 - 0.233	0.172 - 0.214	0.131 - 0.194	min_samples_leaf (worse as leaves grow)
kNN	0.233 - 0.237	0.224 - 0.228	0.188 - 0.197	0.155 - 0.171	n_neighbors (worse as k grows)
Kalshi Market	0.213	0.201	0.166	0.126	–

Each cell shows the best and worst CV Brier score across all settings in the grid. Lower is better.